

Smart2Raw

Armazenamento numérico auto-adaptativo

Documento completo: da ideia à arquitetura, ao uso e à implementação

Da ideia simples ao pseudocódigo — com diagramas, exemplos e cálculos

Versão de referência 3.3.5 — Maio de 2026

Sumário

Sumário executivo

O Smart2Raw é uma biblioteca C header-only para armazenar e operar sobre grandes arrays de inteiros usando o menor tipo nativo que comporta o range real dos dados (8, 16, 32 ou 64 bits, com ou sem sinal). A premissa é simples: a maioria dos dados do mundo real usa apenas uma fração dos bits que normalmente alocamos para eles. Em vez de comprimir e descomprimir, o Smart2Raw classifica os dados uma vez e opera diretamente no formato compacto, com acesso tipado em $O(1)$.

Este documento é o material completo do projeto, organizado em oito partes: começa pela ideia em linguagem acessível e exemplos do dia a dia; passa pela arquitetura e pelo funcionamento interno com diagramas; descreve as inovações e o raciocínio por trás delas; mostra onde usar e os benefícios medidos; reúne a referência técnica; traz o anexo de implementação e pseudocódigo; e fecha com os mecanismos de ganho medidos, o posicionamento e o potencial de economia. A versão de referência é a 3.3.5, em que toda a biblioteca — inclusive os módulos de analytics e de largura por bloco (PFOR, com sinal e soma por bloco acelerada por SIMD) — vive num único arquivo header-only.

Princípio central: classifique uma vez, opere sempre no formato mais compacto que preserva todos os valores. Sem descompressão, sem bit-packing manual, sem dependências externas.

Honestidade de escopo: o Smart2Raw é uma camada de armazenamento e varredura de inteiros. Não é compressor de propósito geral, não armazena floats e não executa multiplicação de matrizes. Os ganhos aparecem em leitura, agregação e movimentação de dados — não no cálculo denso de álgebra linear.

Parte I — A ideia, de forma simples

O que o Smart2Raw faz e por que funciona, com exemplos que qualquer pessoa entende.

1. A ideia central: o que o Smart2Raw é (e o que não é)

Considere guardar a temperatura de um sensor: 25 °C. Em binário, 25 é 11001 — cinco bits bastam. Ainda assim, é comum declará-la como um inteiro de 64 bits, desperdiçando 59 bits (92%) só para armazenar zeros. Quando isso se multiplica por milhões de leituras, o desperdício deixa de ser teórico: ele consome memória, polui o cache, gasta banda e energia movendo bits inúteis.

O Smart2Raw parte da observação de que idades cabem em 8 bits, códigos HTTP em 10, contadores de página em 14, e assim por diante. A biblioteca examina o range real de um conjunto de dados e escolhe a menor classe nativa que o comporta. A classificação custa uma varredura única; a partir daí, todo acesso é uma leitura tipada direta, sem máscara nem deslocamento de bits.

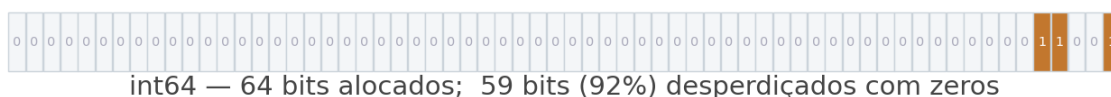
Definir as fronteiras é tão importante quanto descrever as capacidades, e foi um princípio que guiou todo o trabalho:

- **Não é compressão de propósito geral.** Ele escolhe um único tipo para o array inteiro, pelo maior valor presente. Um outlier grande arrasta toda a coleção para uma classe maior.
- **Não armazena floats.** É uma estrutura de inteiros. Dados de ponto flutuante precisam ser quantizados antes (e o Smart2Raw não faz a quantização).
- **Não faz álgebra linear densa.** Não há multiplicação de matrizes nem produto-ponto de propósito geral. As operações são reduções e transformações elemento-a-elemento (soma, mínimo, máximo, contagem, filtros, escalares).

2. Por que cabe menos: o desperdício de bits

Todo o ganho nasce de uma observação simples. Guardar o valor 25 exige cinco bits (11001). Declará-lo como inteiro de 64 bits aloca 64 bits — 59 deles (92%) são zeros que não carregam informação.

Guardar o valor 25 = binário 11001 (5 bits bastam)



← **Smart2Raw S2R_8: apenas 8 bits (1 byte)**

Figura 1 — O mesmo valor em int64 (59 bits desperdiçados) e em S2R_8.

O Smart2Raw não comprime: escolhe a classe que comporta o range real e armazena o valor nela. O acesso continua sendo uma leitura nativa — sem shift nem máscara, como haveria num bit-packing manual.

3. Densidade de cache e banda de memória

O desperdício de bits não é só memória parada: ele se propaga para o cache e para a banda. Menos bytes por valor significam mais valores por linha de cache, e uma varredura encontra muito menos cache miss.

Uma linha de cache de 64 bytes



Figura 2 — Quantos valores cabem numa linha de cache de 64 bytes.

Cálculo concreto: 1 milhão de leituras de temperatura (0 a 60 °C)

```
Convencional (int64): 1.000.000 x 8 bytes = 8.000.000 B ~ 7,63 MiB
Smart2Raw (u8):      1.000.000 x 1 byte  = 1.000.000 B ~ 0,95 MiB
Economia: 7 MB (87,5%)
```

Linha de cache de 64 B: int64 -> 8 valores ; u8 -> 64 valores (8x)

Somar 1M valores (limitado por banda, a ~10 GB/s):

int64: move 8 MB ~ 0,8 ms ; u8: move 1 MB ~ 0,1 ms -> ~8x

Esse fator de 8 não é teórico: foi o que medimos em reduções de mínimo e máximo (8 a 10x ao habilitar a vetorização), porque, fora do cache, o tempo é dominado pela quantidade de bytes movidos.

4. Exemplos didáticos: o tradicional versus o nosso

Os exemplos a seguir usam números do dia a dia e contas que qualquer pessoa confere de cabeça. Em todos, a resposta é exatamente a mesma nas duas abordagens — o que muda é quanta memória se gasta para guardar os dados. A regra mental é uma só: pergunte “qual o maior valor que vou guardar?” e use o menor tipo que o comporta. O Smart2Raw faz isso por você, automaticamente.

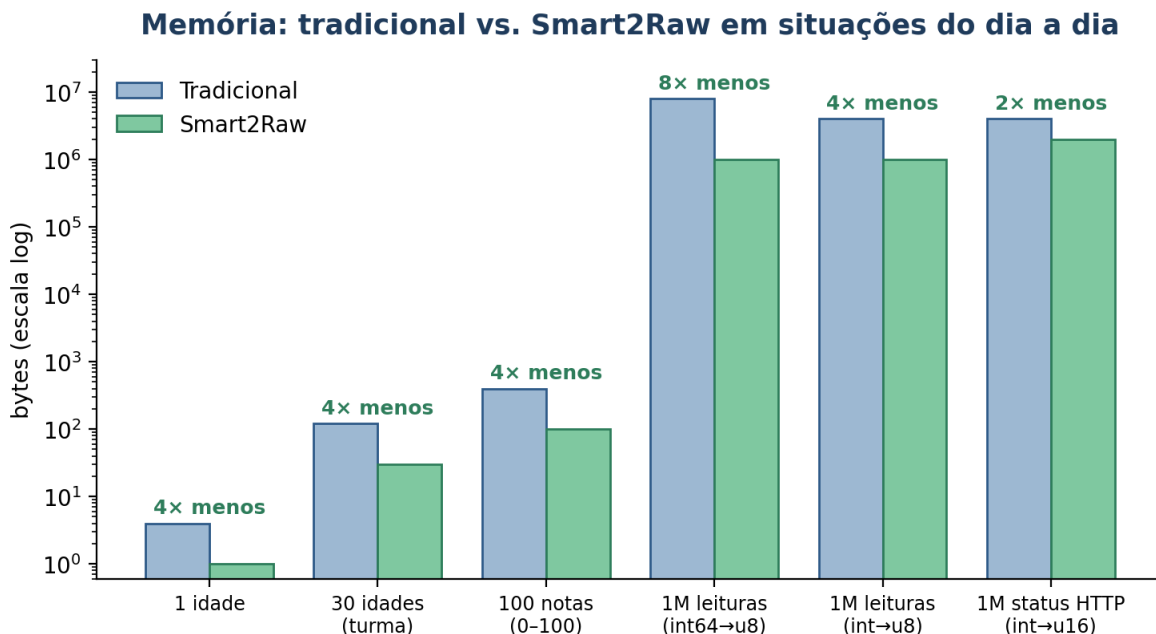


Figura 3 — Memória: tradicional vs. Smart2Raw em situações do dia a dia.

4.1 Guardar uma idade (37 anos)

Uma idade vai de 0 a ~120. O valor 37, em binário, é 100101 — seis bits bastam. O reflexo comum é declará-la como um inteiro de 4 bytes.

```
37 em binario = 100101 (6 bits bastam)
Tradicional (int): 4 bytes = 32 bits -> 26 bits desperdicados
Smart2Raw (u8): 1 byte = 8 bits
Mesma idade guardada; 4x menos memoria.
```

4.2 As idades de uma turma (30 alunos)

Para a média, soma-se tudo e divide-se por 30 — a conta é idêntica nas duas abordagens. A diferença é só o espaço ocupado pela lista.

```
Tradicional (int[30]): 30 x 4 bytes = 120 bytes
Smart2Raw (u8[30]): 30 x 1 byte = 30 bytes
media = soma / 30 (mesmo resultado) -> 4x menos memoria
```

4.3 As notas de uma prova (0 a 100)

Notas inteiras de 0 a 100 cabem folgadoamente em um byte (que vai até 255). Maior nota, menor nota e média saem idênticas.

```
Tradicional (int[100]): 400 bytes
Smart2Raw (u8[100]): 100 bytes
maior, menor e media: resultados identicos -> 4x menos memoria
```

Antes de seguir, vale ver quantos bits cada coisa do cotidiano realmente precisa — quase tudo cabe em uma ou duas classes pequenas:

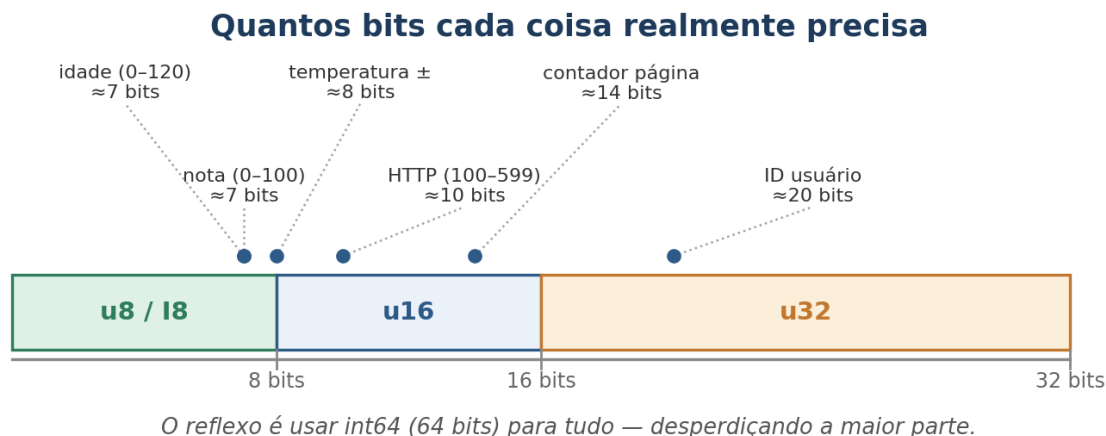


Figura 4 — Quantos bits cada valor do dia a dia realmente precisa.

4.4 Temperaturas com valores negativos (−10 a 35 °C)

Quando há negativos, usa-se uma classe com sinal. A faixa de −10 a 35 cabe num único byte com sinal (I8, que vai de −128 a 127). A soma é a mesma — e, ao contrário das primeiras versões da biblioteca, agora é feita corretamente respeitando o sinal.

```
Faixa -10..35  cabe em I8 (-128..127): 1 byte com sinal
Tradicional (int): 4 bytes    ;   Smart2Raw (I8): 1 byte
soma: (-10) + (-3) + 20 + 35 = 42   (correta nos dois) -> 4x menos
```

4.5 Um contador que cresce (curtidas de um post)

Aqui aparece a vantagem mais sutil. As curtidas começam pequenas e cabem num byte (até 255). O problema é o que acontece ao passar de 255.

```
Tradicional ingenuo (u8): 256 da a volta para 0   (ERRO silencioso)
Tradicional seguro (int): já gastava 4 bytes desde a primeira curtida
Smart2Raw: cresce sozinho u8 -> u16 quando precisa,
           sem truncar e sem gastar memória antes da hora
```

É o melhor dos dois mundos: começa econômico como o byte e fica seguro como o inteiro grande, sem o programador ter de adivinhar o tamanho certo na frente.

4.6 Honestidade: quando NÃO há economia

O ganho não é mágica — ele existe quando os valores são pequenos. Se o valor é grande por natureza, o tipo ótimo já é grande, e não há o que economizar.

```
Total de compra R$ 1.234,56 = 123.456 centavos -> precisa de u32
Aqui o tipo ótimo JA É 32 bits: não há economia vs um int de 32 bits.
A economia aparece com valores pequenos (idades, notas, sensores, status).
```

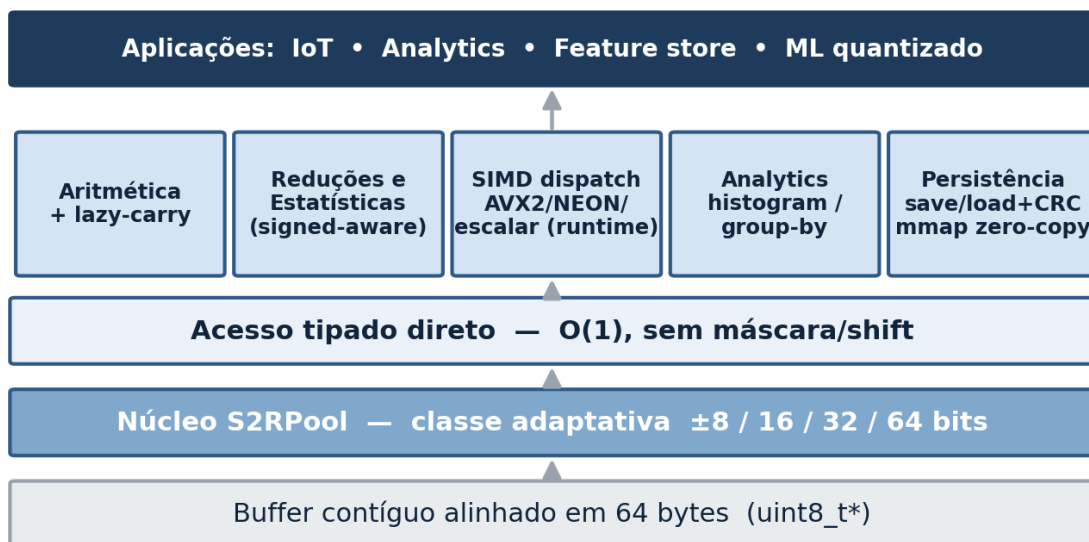
Em uma frase: para a maioria dos dados do mundo real — que são pequenos — guardar no menor tipo que cabe economiza memória, cache e banda sem mudar nenhuma resposta; quando os dados são grandes, o Smart2Raw simplesmente usa o tipo grande, como qualquer um faria.

Parte II — Arquitetura e funcionamento interno

Como o sistema é estruturado e como a conta é feita por dentro da máquina.

5. Arquitetura em camadas

O Smart2Raw é organizado em camadas finas sobre um único buffer de bytes. Cada camada faz uma coisa e empilha sobre a anterior, o que mantém o sistema auditável e permite remover peças inteiras (arquivos, mmap, SIMD) em builds enxutos sem tocar no núcleo.



Header-only • zero dependências externas • gates: S2R_NO_STDIO / _MMAP / _SIMD

Figura 5 — Arquitetura em camadas do Smart2Raw.

De baixo para cima: um buffer contíguo alinhado em 64 bytes hospeda os dados; o núcleo S2RPool decide a classe (largura em bits, com ou sem sinal) e a torna o tipo do array; o acesso tipado direto transforma a leitura do elemento i numa única indexação $O(1)$. Sobre essa base, módulos independentes oferecem aritmética (inclusive lazy-carry), reduções e estatísticas cientes do sinal, o despacho SIMD em runtime, as operações de analytics e a persistência (serialização portátil com CRC e mmap zero-copy). No topo ficam as aplicações.

Decisão de projeto: nada acima do núcleo conhece o layout físico além de “classe + ponteiro”. É isso que permite que a mesma soma rode sobre um pool em RAM ou sobre um arquivo mapeado, e que o SIMD escolha o caminho da CPU sem o código de aplicação saber.

6. Estruturas de dados e layout de memória

A unidade central é o S2RPool: metadados mínimos apontando para um buffer contíguo. Não há cabeçalho por elemento nem ponteiros internos — o pool é, na prática, um vetor C cujo tipo foi decidido em runtime. Ler o elemento i resume-se a reinterpretar o buffer como ponteiro do tipo

correto e indexar; comparado ao bit-packing manual (que exige shift e máscara a cada acesso e não vetoriza bem), o acesso tipado é uma única operação que deixa o autovetorizador trabalhar.

S2RPool: metadados + buffer alinhado

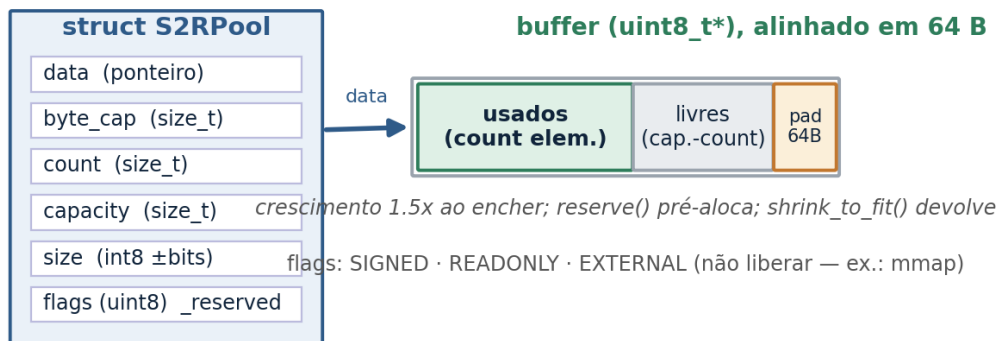


Figura 6 — Metadados do S2RPool e o buffer alinhado.

Os campos do S2RPool:

Campo	Tipo	Significado
data	uint8_t*	ponteiro para o buffer alinhado (alias-safe)
byte_cap	size_t	bytes efetivamente alocados (com padding)
count	size_t	número de elementos em uso
capacity	size_t	capacidade em elementos
size	int8_t	largura em bits; negativo = com sinal (±8/16/32/64)
flags	uint8_t	SIGNED · READONLY · EXTERNAL
_reserved	uint16_t	preenchimento para alinhamento

Estruturas auxiliares completam a arquitetura, cada uma envolvendo um pool ou descrevendo-o:

Estrutura	Papel
S2RInfo	diagnóstico: contagem, capacidade, bytes, eficiência de memória
S2RMap	handle de mmap: pool + base/len/fd do mapeamento (libera por munmap)
S2RTracked	pool com mínimo/máximo embutidos, atualizados em O(1) no push
S2RDeferred / S2RDeferredSigned	sessão lazy-carry: acumula a magnitude da cadeia até o commit

7. Modelo de memória e ciclo de vida

A gestão de memória é explícita e previsível, o que importa tanto no servidor quanto no microcontrolador.

- **Alocação alinhada.** O buffer é alocado em fronteira de 64 bytes (aligned_alloc no C11, _aligned_malloc no Windows, fallback manual em outros), favorecendo linha de cache e cargas SIMD.

- **Crescimento.** Quando o pool enche, a capacidade cresce por fator 1,5× (mais econômico em memória que 2×). `s2r_reserve` pré-aloca; `s2r_shrink_to_fit` devolve o excesso.
- **Posse (ownership).** A flag `EXTERNAL` indica que os dados são de terceiros (por exemplo, um mapeamento): `s2r_pool_free` não os libera. A flag `READONLY` rejeita escritas. Um pool de `mmap` carrega ambas e é liberado por `s2r_map_close` (`munmap`), nunca por `free`.
- **Promoção e rebaixamento.** `promote/demote` e `fit_class` realocam o buffer e convertem os elementos para a nova classe; a capacidade passa a igualar a contagem (ajuste exato).

8. A decisão de classificação

Classificar é encontrar a menor classe que comporta o range. Para valores sem sinal, basta o máximo; para valores com sinal, é preciso o intervalo completo (mínimo e máximo).

Faixa de valores	Classe (sem sinal)	Classe (com sinal)
cabe em 8 bits	S2R_8 (0–255)	S2R_I8 (–128...127)
cabe em 16 bits	S2R_16 (0–65.535)	S2R_I16 (–32.768...32.767)
cabe em 32 bits	S2R_32 (0–~4,29×10 ⁹)	S2R_I32 (±~2,15×10 ⁹)
maior	S2R_64	S2R_I64

A classificação de um array é uma única varredura $O(n)$ que encontra o(s) extremo(s). A partir daí, a classe vira o tipo do pool e o byte size serve, de quebra, como uma impressão digital barata da magnitude dos dados — varrer só esses bytes em milhares de colunas diz quais são minúsculas sem tocar nos dados.

9. Microprocessamento: como a conta é feita por dentro

Uma CPU moderna soma usando registradores SIMD — na geração AVX2, de 256 bits. O que muda entre o convencional e o Smart2Raw é quantos valores cabem nesse registrador e como a soma é feita.

Microprocessamento: somar valores num registrador SIMD de 256 bits

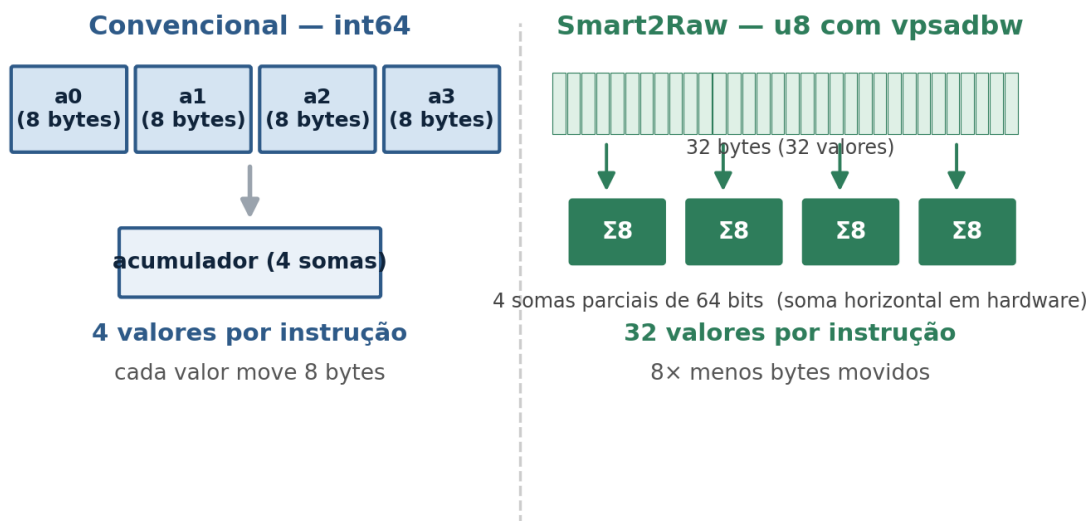


Figura 7 — Somar no registrador SIMD: int64 (4 valores) vs u8 com vpsadbw (32 valores).

9.1 Como é feito hoje (convencional, int64)

Cada valor ocupa 8 bytes; um registrador de 256 bits comporta quatro int64. A CPU carrega quatro, soma ao acumulador e repete; ao final, soma horizontalmente as quatro lanes. São quatro valores por instrução, e cada valor obriga a mover 8 bytes. Pior: se você guarda bytes mas soma num acumulador de 64 bits, o compilador precisa promover (widening) 8 → 64, processando poucos elementos por instrução. Foi por isso que a soma escalar de bytes ficou presa em ~4 GB/s mesmo em cache — limitada por cálculo, não por banda.

9.2 Como o Smart2Raw faz (u8 com vpsadbw)

A instrução `vpsadbw` calcula, para cada bloco de 8 bytes, a soma das diferenças absolutas contra um vetor. Usando zero como referência, vira uma soma horizontal de 8 bytes feita em hardware. Um registrador de 256 bits traz 32 bytes, produzindo quatro somas parciais de 64 bits numa única instrução: 32 valores por instrução, sem widening, movendo 8x menos bytes.

9.3 Exemplo passo a passo (oito valores)

Some [25, 30, 40, 10, 5, 60, 20, 50] — a resposta é 240. Veja como cada abordagem chega lá:

```
Em memoria: int64 -> 64 bytes ; u8 -> 8 bytes (19 1E 28 0A 05 3C 14 32)
```

```
Convencional (int64, AVX2): 2 cargas (4+4 lanes), soma + horizontal.
move 64 bytes ; ~4 valores por instrucao
```

```
Smart2Raw (u8, vpsadbw): 1 carga traz os 8 bytes.
vpsadbw(bytes, 0) = 25+30+40+10+5+60+20+50 = 240 (1 instrucao)
move 8 bytes ; soma horizontal gratuita (em hardware)
```

Resultado idêntico (240); custo 8x menor em memória, soma horizontal resolvida pela própria instrução. Medimos esse caminho de 2,8x (RAM) a 10x (cache) mais rápido que o autovetorizador para u8.

9.4 O dispatch que mantém um único binário

A função AVX2 é compilada com atributo de alvo, então existe no binário sem se compilar tudo com `-mavx2`. Em runtime, consulta-se a CPU: havendo AVX2, usa-se `vpsadbw`; no ARM, um caminho NEON equivalente (somas pareadas com widening até 64 bits); senão, escalar. O mesmo executável serve do servidor à borda.

10. Cálculos lado a lado: convencional vs. Smart2Raw

Contas simples para tornar a diferença palpável. Em todas, o resultado numérico é o mesmo; muda quanto trabalho e quanta memória são gastos.

10.1 Contar quantas leituras passam de 30 (entre 1 milhão)

```
Convencional (int64): le 8 MB para varrer 1M valores
Smart2Raw (u8):      le 1 MB para varrer 1M valores
Mesma resposta (a contagem); 8x menos banda de leitura.
```

10.2 A soma como somatório de bytes (o detalhe do hardware)

```
Laco convencional: s=0; s+=25; s+=30; ... (8 passos dependentes)
vpsadbw (1 passo): Sigma(25,30,40,10,5,60,20,50) = 240
```

10.3 Um cálculo que aparece em IA: correção de zero-point (int8)

Na quantização assimétrica da inferência int8, a saída de $C = A \cdot B$ precisa de termos de correção que são somas de linha e coluna sobre os inteiros — uma redução, não um matmul. Para A de 4096×4096 , calcula-se a soma de cada uma das 4096 linhas de 4096 bytes.

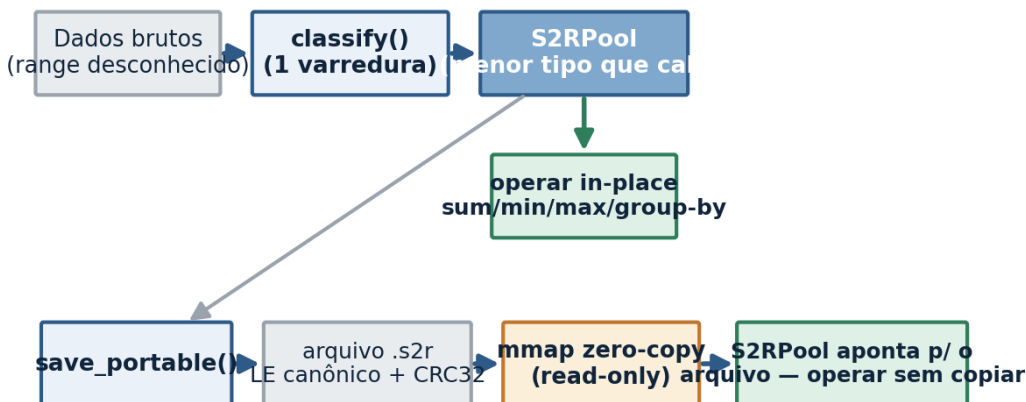
```
row_sum[i] = soma dos 4096 bytes da linha i, i = 0..4095
Convencional (escalar): ~16,7 M somas de byte
Smart2Raw (vpsadbw):    128 instrucoes SAD por linha
Medido: 2,3x mais rapido, resultado identico.
```

Conclusão honesta: o Smart2Raw não faz o produto de matrizes (território de kernels de GEMM), mas acelera as reduções de correção que acompanham todo matmul int8 assimétrico — ganho real e localizável dentro da pilha de IA.

11. Fluxo de dados e persistência

O ciclo de vida segue a filosofia do projeto: classificar uma vez, operar sempre no compacto. Os dados brutos passam por uma única varredura, viram um S2RPool no menor tipo que cabe, e a partir daí toda operação é direta — em RAM ou em disco.

Fluxo de dados: classificar uma vez, operar no compacto



Sem descompressão em nenhuma etapa

Figura 8 — Da ingestão à operação, em RAM e via mmap zero-copy.

Na persistência, a serialização grava um formato little-endian canônico com CRC32, portátil entre arquiteturas. O mmap mapeia o arquivo read-only e faz o pool apontar para dentro dele: as agregações rodam sobre o armazenamento sem copiar o dataset para a RAM — o caso de uso decisivo na borda.

12. Formato em disco (.s2r)

O arquivo é autodescritivo e portátil. O layout fixo de 16 bytes de cabeçalho permite que o mmap localize o payload por um offset constante.

Formato em disco .s2r (little-endian canônico)

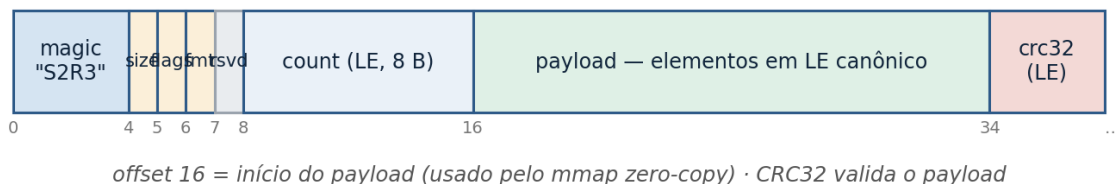


Figura 9 — Layout de bytes do arquivo .s2r.

Offset	Tam.	Campo	Conteúdo
0	4	magic	"S2R3" (0x33335253), little-endian
4	1	size	classe (int8; negativo = com sinal)
5	1	flags	S2RFlags

Offset	Tam.	Campo	Conteúdo
6	1	fmt	versão do formato (1)
7	1	rsvd	reservado (0)
8	8	count	número de elementos (LE)
16	count×eb	payload	elementos em LE canônico
16+n	4	crc32	CRC32 IEEE 802.3 do payload (LE)

Em host little-endian, o payload em disco coincide byte-a-byte com a memória, e arquivos antigos (escritos em LE) continuam legíveis. Em big-endian, a leitura converte; o CRC é validado sobre os bytes canônicos antes de qualquer conversão.

13. Largura bidirecional (auto-cura)

A largura adapta-se nas duas direções: o push adaptativo promove a classe quando um valor não cabe; removido o outlier, o ajuste de classe rescaneia o range e rebaixa o pool, reciclando memória.

Largura bidirecional (auto-cura)



cresce no ingest, encolhe na limpeza — recicla memória

Figura 10 — A largura cresce no ingest e encolhe na limpeza.

Isso responde à fragilidade do modelo: sem a auto-cura, um único outlier transitório degradaria permanentemente a economia da coleção. Um pool com intervalo embutido (S2RTracked) mantém mínimo e máximo em $O(1)$, evitando rescans quando o estado está limpo.

Parte III — O que inovamos, como e por quê

Cada inovação sobre a base original, o problema que resolve e a evidência medida.

14. As inovações e o raciocínio por trás

Esta parte descreve, de forma discursiva, cada inovação introduzida sobre a base original, o problema que ela resolve e a evidência que a sustenta. Todas as afirmações de desempenho vêm de medições reais conduzidas durante o desenvolvimento.

14.1 Correções de correção: agregações com sinal e desvio-padrão

A base anunciava inteiros com sinal e estatísticas como recursos de destaque, mas eles não compunham entre si. As funções de mínimo, máximo, variância e contagem interpretavam os bytes como sem sinal — então, num pool com sinal contendo negativos, produziam silenciosamente resultados errados. Num conjunto de -100 a 100, o mínimo retornava 0, o máximo retornava 255 (o -1 lido como byte sem sinal) e a variância vinha quase três vezes maior que a correta.

A correção foi prover versões cientes do sinal (mínimo, máximo, variância, desvio-padrão, contagens e somas condicionais), que leem cada elemento com a interpretação correta. Separadamente, o desvio-padrão tinha um bug grave: o sqrt por Newton-Raphson com chute inicial igual à própria variância não convergia para variâncias grandes — num caso de variância da ordem de $1,3 \times 10^{18}$, devolvia um valor com erro relativo de cerca de um milhão de vezes. Foi substituído por uma raiz quadrada robusta que normaliza a entrada por potências de quatro antes de iterar, sem depender de `math.h`.

Por que importa: recursos que se anunciam como prontos precisam compor. Um sistema que dá a resposta errada em silêncio é pior do que um que recusa — por isso a prioridade foi tornar o caminho com sinal correto e cobri-lo com testes que provam o erro antigo e a correção nova.

14.2 Aritmética lazy-carry

As operações escalares nativas (somar, multiplicar) faziam wraparound silencioso: somar 60 a um valor 250 num pool de 8 bits resultava em 54, sem aviso. A inovação foi uma aritmética que adia a decisão de tipo. Em vez de truncar, ela calcula a magnitude do resultado e promove a classe do pool apenas quando — e o quanto — for necessário.

A versão de operação única promove na hora se o resultado estourar. A versão de sessão deferida é o conceito de lazy-carry no sentido literal: encadeia várias operações, acumula o pior caso de magnitude da cadeia inteira e promove uma única vez no commit. Como soma e multiplicação por escalar são monotônicas em magnitude no caso sem sinal, dimensionar pelo valor final garante que nenhum intermediário estoura. A variante com sinal rastreia o intervalo completo (mínimo e máximo), tratando corretamente o caso do multiplicador negativo, que inverte os limites. Estouros que excedem 64 bits são recusados explicitamente, em vez de corromper os dados.

Por que importa: transforma uma fonte clássica de bugs (overflow silencioso) numa propriedade do sistema (a largura adapta-se ao resultado), reaproveitando a mesma filosofia adaptativa do projeto.

14.3 SIMD com dispatch em runtime

Duas descobertas guiaram esta inovação. Primeiro, a alavanca mais barata não era código, era flag de compilação: ao trocar `-O2` por `-O3 -march=native`, as reduções de mínimo e máximo ficaram de oito a dez vezes mais rápidas sem alterar uma linha, porque o autovetorizador só vetoriza reduções com `-ftree-vectorize`. Segundo, mesmo com o autovetorizador ligado, a soma de tipos pequenos era lenta: somar bytes num acumulador de 64 bits força um widening custoso. Uma soma AVX2 explícita com a instrução de soma horizontal de bytes (`vpsadbw`) foi de duas a três vezes mais rápida em memória e cerca de dez vezes em cache.

Para que um único binário sirva do servidor x86 ao dispositivo de borda, o despacho é feito em runtime: a função AVX2 é compilada com atributo de alvo (não exige `-mavx2 global`) e só é chamada se a CPU reportar suporte; caso contrário, cai no caminho escalar. No ARM, um caminho NEON análogo (cadeia de somas pareadas com widening até 64 bits) cobre o mesmo papel. O resultado escalar e o vetorizado são idênticos, verificados em tamanhos variados, inclusive não-múltiplos da largura do vetor.

14.4 I/O portátil: endianness canônica e CRC32

A serialização original gravava bytes nativos — inclusive o número mágico e a contagem. Um arquivo escrito em little-endian não carregava corretamente em big-endian, apesar de existirem macros de detecção de endianness que nunca eram usadas. Além disso, havia um código de erro de corrupção que nada validava.

A nova serialização grava sempre em little-endian canônico (montando os bytes explicitamente por deslocamento, independente do host) e anexa um CRC32 IEEE 802.3 do payload. Em host little-endian, o formato em disco coincide byte-a-byte com o antigo, preservando a compatibilidade de leitura; em big-endian, faz a conversão. O CRC, validado contra o vetor de teste padrão, permite detectar corrupção de um único byte na carga.

14.5 mmap zero-copy

Para o cenário de borda, em que datasets grandes precisam ser lidos do armazenamento sem caber na RAM, a biblioteca passou a oferecer mapeamento de memória. Um arquivo é mapeado read-only e o pool aponta diretamente para dentro do mapeamento — sem cópia, paginado sob demanda pelo kernel. Todas as operações de leitura e agregação funcionam direto sobre o arquivo mapeado. Num host big-endian, o mapeamento é convertido in-place via cópia-na-escrita (`MAP_PRIVATE`), o que mantém o arquivo em disco intacto.

14.6 Largura bidirecional (auto-cura)

O push auto-adaptativo já fazia a largura crescer conforme os dados chegavam. Faltava o caminho inverso. Com a remoção de elementos e uma operação de ajuste de classe que rescaneia o range e rebaixa o pool para a menor classe que ainda comporta os dados, a largura passou a adaptar-se nas duas direções: cresce no ingest, encolhe na limpeza. Removido o outlier que havia forçado o pool a 64 bits, o ajuste o devolve a 8 bits e recicla a memória. Um pool com intervalo embutido mantém o mínimo e o máximo em tempo constante a cada inserção, evitando rescans desnecessários.

Por que importa: resolve diretamente a fragilidade do modelo — a de que um único outlier degrada a economia de toda a coleção de forma permanente.

14.7 Group-by direto no formato compacto

Para dar dente à filosofia de operar sem descomprimir, foram adicionadas operações de agregação por grupo sobre chaves de 8 bits (até 256 grupos): o histograma (equivalente a um GROUP BY chave COUNT(*)) e a soma por grupo (GROUP BY chave SUM(valor)). O histograma usa quatro tabelas parciais para quebrar a cadeia de dependência de leitura-modificação-escrita que serializa incrementos repetidos no mesmo bin.

A caracterização empírica foi reveladora: em dados uniformes, o truque de quatro tabelas não ajuda (não há repetição que serialize); em dados enviesados — o caso comum em analytics, com distribuições do tipo Zipf ou valor-default dominante — o ganho foi de 3,6 a quase 4 vezes, e a versão de quatro tabelas mostrou-se robusta à distribuição, mantendo desempenho estável onde a ingênua despencava cinco a seis vezes.

14.8 O achado int8: correção de zero-point

Uma conclusão inicial precipitada foi a de que o Smart2Raw seria irrelevante para inferência int8, já que o cálculo pesado é a multiplicação de matrizes, que mora em bibliotecas especializadas. A exploração subsequente desmentiu parcialmente isso. A quantização assimétrica (estilo gemmlowp) exige termos de correção de zero-point que são somas de linhas e colunas sobre os inteiros — uma redução, não um matmul. Exatamente o que a soma AVX2 da biblioteca acelera.

O experimento mediu o cálculo de soma por linha de uma camada 4096×4096 (a correção que acompanha todo GEMM int8 assimétrico): a versão com a instrução de soma horizontal foi 2,3 vezes mais rápida que o laço escalar, com resultado idêntico. A fronteira, portanto, não é onde se imaginava: o Smart2Raw toca o caminho de inferência int8 — não no produto de matrizes, mas no passo de correção que o acompanha.

14.9 Ambiente de teste emulado

Como as inovações de NEON (ARM) e big-endian visam hardware indisponível na máquina de desenvolvimento (x86 little-endian), foram construídos dois emuladores que rodam em qualquer x86 com gcc e exercitam o código real. Para o NEON, um cabeçalho de intrínsecos mínimo e fiel à especificação ACLE permite compilar e executar as funções NEON reais, validando índices de lane, a cadeia de widening e o tratamento de resto. Para o big-endian, a leitura nativa de um host BE é simulada de forma determinística: lê-se o arquivo canônico, aplica-se a função de swap real da biblioteca e lê-se cada elemento com montagem big-endian explícita, comparando com os valores originais. Tudo reproduzível por um único script.

Honestidade: emulação valida a lógica do código, não o silício. Recomenda-se um build final em ARM real (ou qemu) e, idealmente, num host big-endian, antes de uso crítico. O fallback escalar e a carga por cópia garantem correção caso os caminhos otimizados sejam desativados.

Parte IV — Onde usar e benefícios esperados

O perfil de carga ideal, os domínios, os ganhos medidos e os limites honestos.

15. O perfil de carga ideal

O Smart2Raw rende quando o padrão de acesso é ler e agregar muitos valores em sequência — varreduras, somas, mínimos e máximos, contagens, consultas por faixa, group-by. Rende menos em acesso aleatório intenso e nada em cálculo denso. O ganho costuma ser proporcional à redução de bytes movidos: quando a operação é limitada por banda de memória, encolher o tipo encolhe o tempo.

16. Domínios de aplicação

Domínio	Por que encaixa	Operações típicas
IoT / sensores / borda	Streams de 8–16 bits, RAM minúscula; mmap lê históricos do flash sem carregar tudo	ingest adaptativo, agregação, mmap
Telemetria / logs / métricas	Status codes, latências, contadores; consultas são analytics puro	count por faixa, soma, group-by
Analytics colunar leve	Colunas inteiras grandes; operação limitada por banda	sum, filtros, histograma
Feature stores tabulares	Categorias, buckets, IDs, flags de range pequeno	scan, group-by, estatísticas
Pré-processamento de ML	Histogramas, contagem de categorias, estatísticas por coluna	histogram, group_sum
Pesos quantizados (int8)	mmap zero-copy do artefato; correção de zero-point	mmap, soma por linha/coluna

17. Benefícios esperados (valores medidos)

Os números abaixo são medições reais do ambiente de desenvolvimento, não estimativas de marketing. O ganho exato depende do hardware, do tamanho do dado relativo ao cache e da distribuição.

Benefício	Magnitude medida	Condição
Memória vs int64	75–87,5% menos	dados que cabem em 8–16 bits
min / max	8–10× mais rápido	só ligando -O3 -march=native (-free-vectorize)
soma u8 (SIMD vpsadbw)	2,8× (RAM) a 10× (cache)	vs autovetorizador
soma u16 (SIMD)	1,4× (RAM) a 4× (cache)	vs autovetorizador
histograma 4-way	3,6–4× (≈1× se uniforme)	dados enviesados (caso comum)
group_sum (u8 → u32)	≈1,78 G linhas/s	direto no formato compacto
correção zero-point int8	2,3×	soma por linha, camada 4096×4096
footprint MCU	≈3,4 KB de código	build sem stdio/mmap/SIMD

18. Casos de uso possíveis

- **Buffer circular de sensor na borda:** ingest adaptativo, transmissão quando cheio, e auto-cura de largura entre janelas.

- **Servidor web contando status codes:** consultas por faixa para taxas de erro 4xx/5xx em tempo real.
- **Faturamento de e-commerce em centavos:** soma, ticket médio e contagem de transações premium sobre milhões de registros.
- **Feature store de ML:** colunas categóricas e numéricas discretizadas, com group-by e estatísticas por coluna.
- **Artefatos de modelo quantizado:** mmap zero-copy dos pesos int8 e aceleração do passo de correção de zero-point.

19. IA e int8: o que realmente entrega

É verdade que int8 traz ganhos enormes em IA, mas convém localizar de onde eles vêm. A maior parte vem da quantização (converter ponto flutuante em int8, reduzindo memória e habilitando GEMM int8) e do próprio GEMM int8 (com acumulação em int32, acelerado por tensor cores ou instruções como VNNI). O Smart2Raw não quantiza nem executa o GEMM — esse é o território de bibliotecas dedicadas.

O que o Smart2Raw entrega na pilha de IA é a fatia de armazenamento e movimentação: o contêiner portátil e o mmap zero-copy para artefatos quantizados, as features e índices inteiros, e — descoberta concreta deste trabalho — a aceleração das somas de correção de zero-point que acompanham todo matmul int8 assimétrico. Em resumo: ganhos reais e localizáveis no IO e nas reduções de apoio, não na multiplicação de matrizes.

20. A questão da baseline (honestidade)

O ganho de memória só existe quando o range real é pequeno, e a baseline de comparação importa. Os 87% são contra inteiros de 64 bits; contra 32 bits, a economia cai pela metade ou mais; contra dados já em int8, não há ganho de memória — apenas o contêiner portátil, o mmap e a agregação acelerada. E porque o pool escolhe um único tipo pelo maior valor, um único outlier de magnitude grande elimina a economia da coleção inteira — fragilidade que a auto-cura de largura mitiga, mas não anula em fluxos com outliers persistentes.

Parte V — Referência técnica

Catálogo de API, complexidade, erros e concorrência.

21. Catálogo de módulos e API

Visão de referência das funções por módulo (nomes reais da biblioteca). Variantes com sufixo `_signed` operam sobre pools com sinal.

Classificação e pool

Função / macro	O que faz
<code>s2r_classify / _signed / _array</code>	menor classe para um valor, valor com sinal, ou array
<code>s2r_classify_signed_range</code>	menor classe com sinal que cobre [min, max]
<code>s2r_pool_init / s2r_pool_free</code>	cria / destrói um pool
<code>s2r_reserve / s2r_shrink_to_fit</code>	pré-aloca / devolve memória não usada
<code>s2r_copy / s2r_fill / s2r_clear</code>	copia, preenche, zera contagem

Acesso e inserção

Função / macro	O que faz
<code>s2r_get / s2r_set (+_signed)</code>	lê / escreve elemento <i>i</i> ($O(1)$)
<code>s2r_as_u8 ... s2r_as_i64</code>	ponteiros tipados para acesso direto
<code>s2r_push / _checked / _saturate</code>	insere; valida; satura no máximo da classe
<code>s2r_push_many</code>	inserção em lote
<code>s2r_push_adaptive (+_signed)</code>	insere crescendo a classe se preciso (nunca trunca)
<code>S2R_FOREACH (+_SIGNED)</code>	macro de iteração

Reduções, estatísticas e filtros

Função / macro	O que faz
<code>s2r_sum / _signed / s2r_sum_fast</code>	soma; soma com sinal; soma acelerada (SIMD)
<code>s2r_min / s2r_max (+ *_signed_val)</code>	extremos (versões com sinal corretas)
<code>s2r_mean / s2r_variance / s2r_stddev (+_signed)</code>	estatísticas
<code>s2r_count_gt / lt / eq / range (+_signed)</code>	contagens por condição
<code>s2r_sum_if (+_signed) / s2r_find</code>	soma condicional / busca

Aritmética, lazy-carry e analytics

Função / macro	O que faz
<code>s2r_add/sub/mul_scalar, xor/and/or/not, transform</code>	operações in-place
<code>s2r_add/mul_scalar_safe (+_signed)</code>	aritmética sem wrap (promove a classe)
<code>S2RDeferred(Signed) begin/add/mul/commit</code>	sessão lazy-carry (uma promoção no fim)
<code>s2r_promote / s2r_demote / s2r_fit_class</code>	muda a classe; <code>fit_class</code> auto-cura
<code>s2r_remove_swap</code>	remoção $O(1)$ sem ordem
<code>s2r_histogram_u8 / s2r_group_sum_u8u32</code>	group-by: COUNT(*) e SUM(val)

Função / macro	O que faz
s2r_blocked_build / get / sum / max / bytes / free	largura por bloco (PFOR): classe por bloco (unsigned)
s2r_blocked_build_signed / get_signed / sum_signed	PFOR com sinal
s2r_blocked_sum_fast	soma por bloco acelerada (SIMD no tipo de cada bloco)

Persistência, diagnóstico e erros

Função / macro	O que faz
s2r_save_portable / s2r_load_portable	I/O LE canônico + CRC32
s2r_map_open / s2r_map_close	mmap zero-copy / liberação
s2r_crc32 / s2r_bswap16/32/64 / s2r_swap_payload	integridade e endianness
s2r_info / s2r_print_info	diagnóstico do pool
s2r_strerror	mensagem para um código de erro

22. Complexidade das operações

Operação	Complexidade	Observação
s2r_get / s2r_set	O(1)	indexação tipada direta
s2r_push	O(1) amortizado	crescimento 1,5×
s2r_push_adaptive	O(1) amort.; O(n) na promoção	reclassifica o buffer quando cresce
sum / min / max / count / find / transform	O(n)	varredura linear
s2r_sum_fast	O(n)	SIMD se disponível (constante menor)
promote / demote / fit_class	O(n) + alocação	converte elementos para a nova classe
save / load_portable	O(n)	CRC também O(n)
map_open (LE)	O(1); O(n) com verify_crc	mmap paginado sob demanda
map_open (BE)	O(n)	swap in-place via COW
histogram / group_sum	O(n)	256 bins

23. Modelo de erros

As funções que podem falhar retornam um código S2RError; s2r_strerror traduz para texto. Operações que apenas leem retornam valores neutros (zero) em entrada inválida.

Código	Valor	Quando ocorre
S2R_OK	0	sucesso
S2R_ERR_NULL	-1	ponteiro nulo
S2R_ERR_OOM	-2	falha de alocação
S2R_ERR_OVERFLOW	-3	estouro inteiro
S2R_ERR_INVALID_SIZE	-4	classe inválida ou pool read-only/incompatível
S2R_ERR_VALUE_TOO_LARGE / _SMALL	-5 / -6	valor fora do range da classe

Código	Valor	Quando ocorre
S2R_ERR_EMPTY	-7	pool vazio
S2R_ERR_IO	-8	erro de leitura/escrita de arquivo
S2R_ERR_CORRUPT	-9	magic inválido ou CRC não confere

24. Concorrência e thread-safety

As garantias são deliberadamente simples e explícitas:

- **Escrita concorrente:** um S2RPool não é seguro para mutações concorrentes (push, set, promote, fit_class alteram estado). Use um pool por thread ou sincronize externamente.
- **Leitura concorrente:** múltiplas threads podem ler com segurança um pool imutável ou READONLY — inclusive um pool de mmap — pois as leituras não tocam estado compartilhado.
- **Tabela de CRC:** s2r_crc32 inicializa sua tabela de 256 entradas na primeira chamada. Em programas multithread há uma corrida benigna (todas as threads computam a mesma tabela); para evitá-la, basta “aquecer” chamando s2r_crc32 uma vez na inicialização.
- **mmap em big-endian:** a conversão in-place ocorre uma única vez dentro de s2r_map_open, antes de o pool ser exposto — logo, leitores subsequentes veem dados já nativos.

Parte VI — Anexo: implementação e pseudocódigo

Como cada peça é implementada, em pseudocódigo próximo do C real.

25. Pseudocódigo das peças centrais

O pseudocódigo é deliberadamente próximo do C real da biblioteca, mas simplificado para foco conceitual.

25.1 Classificação e push auto-adaptativo

A classificação encontra a menor classe que comporta um valor (ou o range de um array). O push adaptativo usa isso para crescer o tipo do pool em vez de truncar.

```

classify(v):                                # sem sinal
    se v <= 0xFF          -> 8
    se v <= 0xFFFF        -> 16
    se v <= 0xFFFFFFFF    -> 32
    senao                  -> 64

push_adaptive(pool, v):
    se v > max_value(pool.size):            # nao cabe na classe atual
        alvo = classify(v)
        se pool.count == 0: pool.size = alvo
        senao: promote(pool, alvo)         # converte buffer p/ classe maior
    push(pool, v)                           # agora cabe, sem truncamento

```

25.2 Aritmética lazy-carry

A versão deferida acumula a magnitude da cadeia e promove uma única vez. A variante com sinal rastreia o intervalo [min, max], pois a multiplicação por escalar negativo inverte os limites.

```

defer_begin(d, pool): d.vmin, d.vmax = range(pool)
defer_add(d, s):      d.vmin += s; d.vmax += s          # com guarda de overflow
defer_mul(d, s):      p1=d.vmin*s; p2=d.vmax*s
                     d.vmin=min(p1,p2); d.vmax=max(p1,p2)
defer_commit(d):
    classe = classify_range(d.vmin, d.vmax)             # cabe a cadeia inteira
    ensure_fits(d.pool, classe)                        # 1 promocao
    para cada op na cadeia: aplica(op)                 # sem intermediario estourando

```

25.3 Soma SIMD com dispatch em runtime

A soma de bytes usa a instrução de soma horizontal (vpsadbw), que soma blocos de 8 bytes em acumuladores de 64 bits. O dispatch escolhe o caminho em runtime.

```

sum_u8_avx2(a, n):
    acc = 0 (4 lanes de 64 bits)
    para i de 0 ate n passo 32:
        acc += sad_epu8(load(a+i), zero)              # soma 32 bytes
    s = horizontal_sum(acc) + cauda_escalar

sum_fast(pool):
    se cpu_tem_avx2() e classe in {8,16}: usa caminho AVX2
    senao se ARM e NEON:                  usa caminho NEON
    senao:                                soma escalar

```

25.4 Largura bidirecional

O ajuste de classe rescaneia o range atual e rebaixa para a menor classe que ainda comporta os dados; a remoção O(1) troca o elemento removido pelo último.

```
remove_swap(pool, i):                # remocao O(1) sem ordem
    pool[i] = pool[count-1]; count -= 1

fit_class(pool):                     # auto-cura: encolhe se possivel
    alvo = classify_range(min(pool), max(pool))
    se largura(alvo) < largura(pool.size):
        reclass(pool, alvo)          # realoca e converte p/ classe menor
```

25.5 I/O portátil com CRC

O formato em disco é sempre little-endian canônico, montado por deslocamento de bytes, seguido de um CRC32 do payload.

```
save_portable(pool, arquivo):
    escreve magic(LE), header[size,flags,fmt], count(LE)
    se host BE e elem>1 byte: payload = swap(copia)    # canoniza p/ LE
    escreve payload
    escreve crc32(payload)

load_portable(pool, arquivo):
    le e valida magic; le header e count(LE)
    le payload; se formato v3.3: verifica crc32 -> erro se difere
    se host BE e elem>1 byte: swap(payload) in-place  # LE -> nativo
```

25.6 mmap zero-copy

O arquivo é mapeado read-only; o pool aponta para dentro do mapeamento (deslocamento do payload), marcado como somente-leitura e externo para que a liberação seja por munmap, não por free.

```
map_open(m, arquivo, verifica_crc):
    fd = open(arquivo, somente_leitura); fsz = tamanho(fd)
    base = mmap(fsz, PROT_READ, MAP_PRIVATE, fd)
    valida magic, classe, count, e que cabe em fsz
    se verifica_crc: confere crc sobre o payload LE
    se host BE e elem>1: mprotect(+WRITE); swap(payload) # COW, arquivo intacto
    m.pool.data = base + 16          # aponta DENTRO do mapeamento
    m.pool.flags |= READONLY | EXTERNAL
map_close(m): munmap(base); close(fd)
```

25.7 Group-by

O histograma usa quatro tabelas parciais para quebrar a dependência read-modify-write; a soma por grupo acumula o valor no bin indexado pela chave.

```
histogram_u8(chaves):                # GROUP BY chave COUNT(*)
    h0,h1,h2,h3 = 4 tabelas de 256 zeros
    para i passo 4: h0[a[i]]++; h1[a[i+1]]++; h2[a[i+2]]++; h3[a[i+3]]++;
    hist[k] = h0[k]+h1[k]+h2[k]+h3[k]

group_sum_u8u32(chaves, valores):    # GROUP BY chave SUM(valor)
    para i: gsum[ chaves[i] ] += valores[i]
```

26. Notas de implementação

- **Alinhamento.** O buffer é alinhado em 64 bytes via alocador alinhado (`aligned_alloc` no C11, `_aligned_malloc` no Windows, fallback manual em outros). As cargas SIMD usam loads não-alinhados (`loadu`), tolerando o deslocamento de payload em mmap.
- **Overflow.** As guardas usam os builtins de overflow do compilador quando disponíveis (`add/mul`) e checagem manual por divisão como fallback portátil.
- **Dispatch.** A função AVX2 leva atributo de alvo, então é compilada com codegen AVX2 mesmo sem `-mavx2` global; a seleção é por consulta de capacidade da CPU em runtime.
- **Cópia-na-escrita.** No caminho big-endian do mmap, o swap in-place ocorre sobre páginas `MAP_PRIVATE`; as alterações não tocam o arquivo em disco — propriedade verificada em teste.
- **Sem sinal vs com sinal.** Como a soma e a multiplicação em complemento de dois são bit-idênticas para ambos na mesma largura, as operações escalares reaproveitam o mesmo núcleo, bastando garantir largura suficiente.

27. Matriz de build e gates de compilação

A biblioteca compila limpa em cinco configurações, da estação de trabalho ao microcontrolador. Os gates permitem builds enxutos:

Cenário	Flags / gates
Servidor x86	<code>-O3 -march=native</code>
Borda x86 (binário único)	<code>-O2 (dispatch AVX2 em runtime)</code>
Borda ARM / sem SIMD	<code>-O2 -DS2R_NO_SIMD</code> (ou build ARM com NEON)
ISO C11 estrito	<code>-O2 -std=c11 -pedantic</code>
MCU / firmware	<code>-Os -DS2R_NO_STDIO -DS2R_NO_MMAP -DS2R_NO_SIMD</code>

Os gates `S2R_NO_STDIO`, `S2R_NO_MMAP` e `S2R_NO_SIMD` removem, respectivamente, a serialização em arquivo, o mapeamento de memória e o despacho SIMD — reduzindo dependências e tamanho. Em qualquer build não-MCU, recomenda-se ao menos `-ftree-vectorize` (ou `-O3`) para destravar a vetorização de mínimo e máximo.

28. Verificação

Na versão 3.3.5, toda a biblioteca vive num único arquivo header-only e há um teste C único e completo (`test_completo.c`, 35 asserções) que exercita todos os módulos: núcleo e push adaptativo, acesso tipado, agregações, estatísticas com sinal, filtros, lazy-carry sem e com sinal, soma SIMD, serialização portátil com CRC, mmap zero-copy, analytics (auto-cura, `S2RTracked`, histograma e `group-by`) e largura por bloco (PFOR). Além dele, a matriz cobre a regressão integrada (35), a compatibilidade retroativa com a suíte original (26), a aritmética com sinal (11), a cópia-na-escrita do mmap (7), os analytics (13), a largura por bloco unsigned (11), o PFOR com sinal e a soma por bloco acelerada (7), além dos emuladores de NEON (26) e big-endian (6). Um único script compila tudo nas cinco configurações (servidor, borda, sem-SIMD, ISO C11 estrito e MCU) e reporta o resultado consolidado: catorze suítes, zero falhas, com o build MCU em torno de 3,4 KB de código. Em complemento, um bug de promoção de classe com o pool vazio (quando o primeiro valor já exigia uma classe maior que a inicial) foi encontrado com o `AddressSanitizer` e corrigido na 3.3.5, com um teste de regressão dedicado para impedir o retorno.

Parte VII — Mecanismos de ganho, posicionamento e economia

Onde o ganho realmente vem (medido), onde vale inserir e o que pode representar em economia.

29. Os quatro mecanismos de ganho

Uma exploração posterior, sempre com medição, mostrou que o ganho do Smart2Raw não é um mecanismo só, são quatro — e a maioria das análises conta apenas o primeiro.

Mecanismo 1 — menos bytes movidos. Tipo menor reduz a banda na proporção dos bytes. É o 4–8× já conhecido, linear.

Mecanismo 2 — o penhasco de cache (superlinear). Quando o tipo menor faz o conjunto de trabalho caber num nível de cache que o int64 não alcança, o ganho passa do linear. Num scan de máximo (sem widening), medimos de 7,6× a 22,8×, com pico justamente na janela em que o u8 mora na L1/L2 e o int64 já transbordou — bem acima do 8× que a banda sozinha previa.

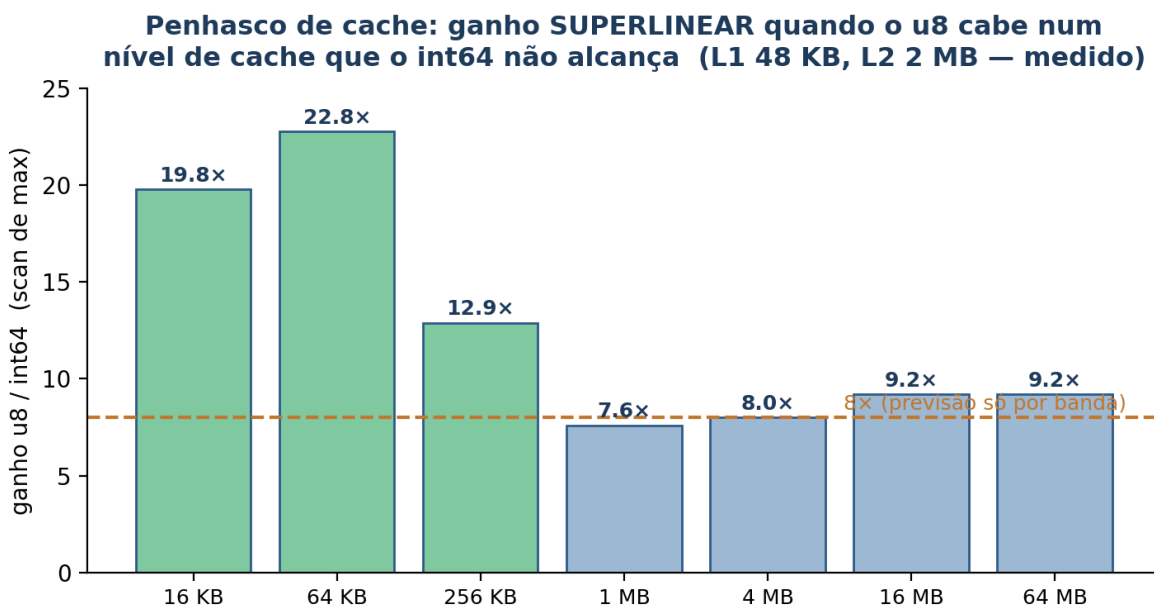


Figura 11 — Ganho superlinear medido: o penhasco de cache (scan de máximo, u8 vs int64).

Mecanismo 3 — inteligência no metadado. A classe (1 byte por coluna) é uma impressão digital do range: serve de zone-map grátis para pular colunas inteiras, e a largura por bloco isola outliers (próxima seção).

Mecanismo 4 — capacidade (mmap > RAM). Não é ganho de tempo, é de viabilidade: operar sobre dados maiores que a RAM, paginados do disco — decisivo para tabelas de embedding, offload de KV-cache e históricos na borda, ainda mais com a memória cara e escassa.

Ressalva honesta: o penhasco beneficia operações sem widening (máximo, mínimo, contagem, filtros) de imediato. A soma de bytes num laço ingênuo pode ser até mais lenta dentro do cache — só captura o ganho com o kernel vpsadbw explícito (que a biblioteca tem). Medir, não assumir.

30. Largura por bloco, zone-maps e o ângulo de IA (medidos)

Três achados medidos durante a exploração, todos com a régua de só afirmar o que foi verificado.

Largura por bloco (PFOR) — conserta a fragilidade. Hoje um único outlier arrastaria o array inteiro para 32 bits. Classificando por bloco (cada bloco escolhe sua largura), o outlier só infla o próprio bloco. Já implementado e testado no header (v3.3.5, tipo S2RBlocked), com e sem sinal: com 0,01% de outliers, a memória medida cai de 16 MB para 4,32 MB — 3,71× recuperado no C real (3,70× na variante com sinal), com soma exata; e ~1,00× sem dano em dados limpos. É a técnica PFOR (Patched Frame-of-Reference) da literatura de compressão.

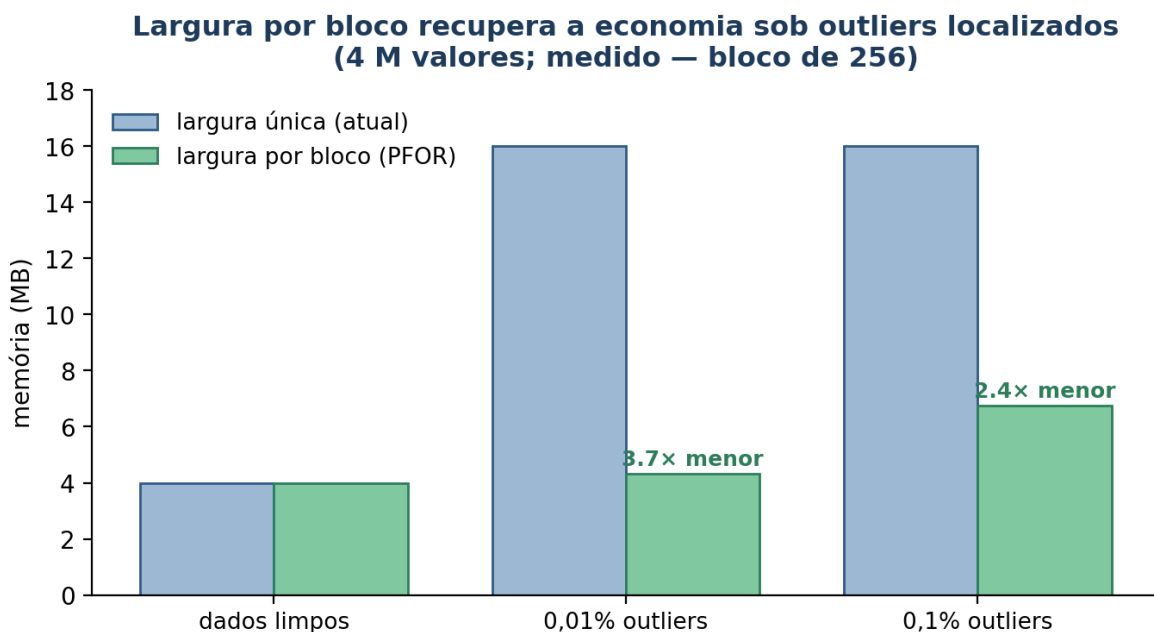


Figura 12 — Recuperação de memória medida com largura por bloco sob outliers localizados.

Zone-map grátis. Numa tabela de 2000 colunas, ler apenas 2000 bytes de metadado (a classe de cada coluna) permitiu pular 98,4% das colunas numa consulta por valor e evitar 94,5% da banda de leitura — skip-scan de banco de dados a custo quase nulo.

Onde a largura por bloco ajuda em IA — tudo medido no código real, com a régua honesta de que, sendo byte-granular, o Smart2Raw não vai abaixo de int8: o ganho aparece quando a largura uniforme segura seria mais larga (tipicamente int16, forçada por outliers) e os outliers estão localizados na memória.

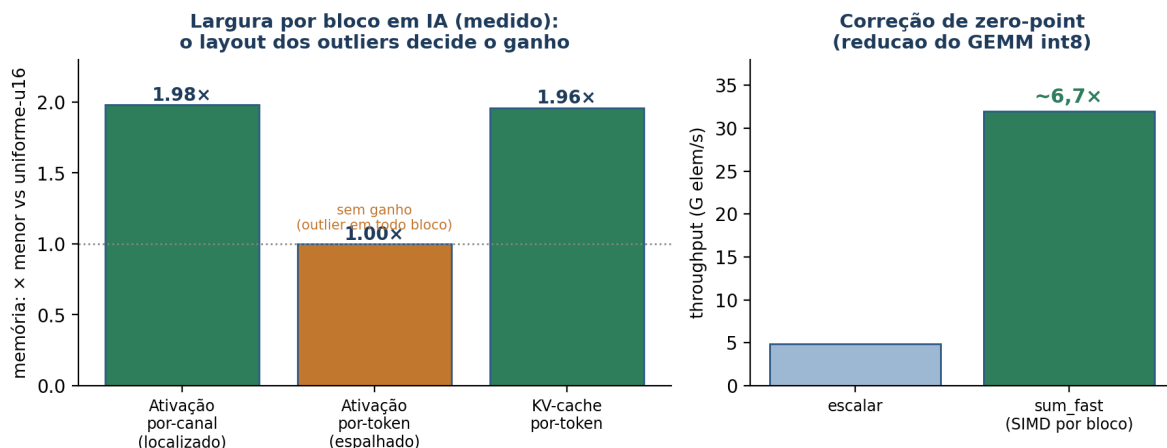


Figura 13 — Largura por bloco em IA (medido): o layout decide o ganho de memória; e a redução de apoio acelera.

Ativações com canais-outlier. Num tensor de ativação 2048×2048 com cerca de 1% de canais-outlier, a largura por bloco deu 1,98× vs. a uniforme-u16 segura, sem perder precisão — desde que o layout seja por canal (cada canal num bloco, com o outlier localizado). No layout por token (o outlier aparece em todos os blocos) o ganho é 1,00×: o layout importa.

KV-cache. Com cerca de 1% de tokens-outlier e blocos por token, a memória do KV caiu de 51,2 MB para 26,1 MB — 1,96× menor, exato.

Correção de zero-point (a redução que acompanha o GEMM int8). A soma por linha do GEMM int8 assimétrico fica rápida mesmo armazenada por bloco: a soma por bloco acelerada (s2r_blocked_sum_fast, que reusa o vpsadbw no tipo de cada bloco) mediu cerca de 6,7× sobre o laço escalar, exata — memória econômica e redução veloz no mesmo passe.

Índices/IDs inteiros. Encolhem 2× quando o vocabulário cabe em 16 bits (token IDs, índices esparsos), mas honestamente 1× quando passa de 16 bits. E continua valendo: nada disso é o GEMM em si — é o armazenamento e as reduções de apoio.

31. Posicionamento e potencial de economia

Onde vale a pena inserir, com a régua honesta. O Smart2Raw é uma camada leve de armazenamento e varredura de inteiros; o ganho aparece quando o dado é inteiro, o range é pequeno e o gargalo é memória/banda — não cálculo denso. E há um ponto que não se pode omitir: a ideia central (usar o menor tipo inteiro) já é prática padrão em bancos colunares (ClickHouse, Parquet, Arrow), então o valor do Smart2Raw está fora desse mundo — onde não há um motor colunar pesado.

Encaixes mais fortes: (1) borda/embarcado/IoT — footprint de ~3,4 KB, sem dependências, adaptativo, mmap; (2) serviços CPU com muitos inteiros guardados ingenuamente em int64/int32 (IDs, contadores, status, códigos) — corta o resident set 2–8×; (3) analytics/métricas leves em memória; (4) como componente (reduções SIMD, group-by) em pipelines maiores.

O ganho por área, sem exagero: em computação geral e cache, o penhasco (mecanismo 2) é o achado transferível; em armazenamento, zone-maps e largura por bloco cortam pegada e banda; em IA, índices/embeddings e quantização por bloco — mas continua não sendo o GEMM, que é de onde vem o grosso do ganho de IA. mmap entrega capacidade, não velocidade.

Potencial de economia (com a régua honesta): não se multiplica o capex de IA pelo speedup. O endereçável é a fatia de memória/banda gasta com dado inteiro pequeno ainda não otimizado. Mas o momento é favorável: estimativas de mercado apontam capex hyperscaler em torno de US\$ 600–725 bilhões em 2026 (cerca de 75% em IA), e a memória virou o recurso escasso — a maior escassez de DRAM em cerca de 15 anos, com preços de contrato de DRAM de servidor subindo perto de 90% num único trimestre e capacidade vendida até o fim de 2026. A alavanca que o Smart2Raw puxa — menos bytes residentes e movidos — é exatamente a que está cara hoje, então cada GB economizado vale mais. Mas o grosso desse capex é GPU para treino e inferência, que o Smart2Raw quase não toca; ele pega a carona macro só parcialmente, no anel de CPU, armazenamento e telemetria ao redor das GPUs.

32. Benefícios aproximados por tipo de uso e economia de servidores

Esta seção reúne, por tipo de uso, o ganho de armazenamento, a velocidade e uma estimativa de economia/capacidade de servidor. Importante: as colunas de armazenamento e velocidade vêm de medições deste projeto; a coluna de servidor é uma estimativa condicional, derivada de um modelo simples (explicado adiante), e depende de quanto da carga real tem o perfil adequado.

Tipo de uso	Armazenamento / memória (medido)	Velocidade (medido)	Servidor / capacidade (estimado)
Telemetria, métricas, logs	75–87% menos vs int64	scans de contagem/faixa 8–22×; histograma 3,6–4×	fleet memória-bound: ~1,3–2× de capacidade
IoT / sensores / borda	75–87% + flash; código ~3,4 KB	agregação 8–10×; mmap evita carregar tudo	MCU/flash menores; mais leituras por dispositivo
Analytics colunar leve	PFOR 3,7× sob outliers; zone-map evita 94,5% da banda	sum_fast 2,8–10×; group_sum ~1,78 G linhas/s	menos nós para o mesmo volume de scan
Feature store / pré-proc. ML	50–87% conforme o range	histograma/group-by 3,6–4×; estatísticas por coluna	mais colunas/features por nó
IA — ativações / KV quantizados	~2× vs uniforme-u16 sob outliers; 0 vs int8 plano	correção de zero-point ~6,7×	mais contexto/tokens por GB de KV (capacidade)
IA — pesos quantizados int8	0 vs int8 (ganho é mmap/portabilidade)	6,7× na redução de apoio	servir modelo > RAM via mmap (capacidade)
Índices / IDs inteiros	2× se vocabulário ≤16 bits; 1× acima	scan e skip por classe	proporcional à fatia de índices
Dados já em int8, floats ou grandes	sem ganho de memória (honesto)	—	só contêiner, mmap e agregação

32.1 Como ler a estimativa de servidor (modelo)

O ganho de servidor não se obtém multiplicando o speedup pelo parque inteiro — seria desonesto. O que se estima é quanto da memória residente é o perfil qualificável (inteiro, range pequeno, ainda não otimizado) e quanto ele encolhe. Sejam f a fração da memória residente nesse perfil e r o fator de redução desse perfil. A mesma RAM passa a comportar mais dado:

```

multiplicador de capacidade = 1 / [ (1 - f) + f / r ]
memória liberada (mesmo dado) = f * (1 - 1/r)

```

Ex.: $f = 50\%$ da RAM no perfil, $r = 6\times$ de reducao

liberada = $0,5 * (1 - 1/6) = 41,7\%$ da memória
 capacidade = $1 / (0,5 + 0,5/6) = 1,71x$ (mesma RAM, ~1,7x mais dado)

Quanto mais dado cabe no MESMO servidor (estimativa pelo modelo)
capacidade = $1 / [(1-f) + f/r]$

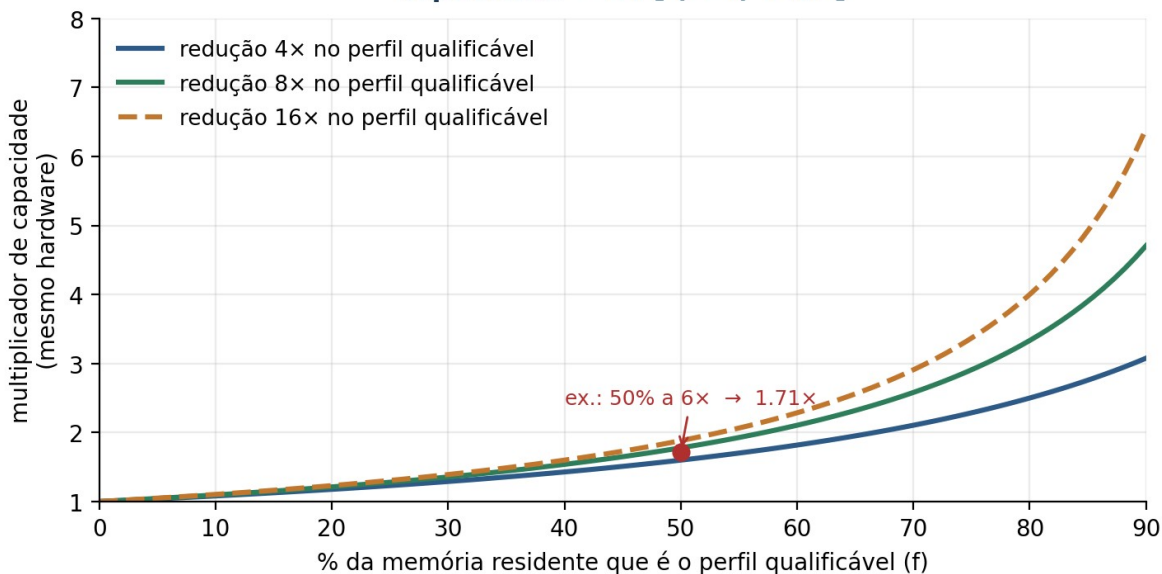


Figura 14 — Multiplicador de capacidade no mesmo hardware, pelo modelo (estimativa).

Fração qualificável (f)	Capacidade a 4x	Capacidade a 8x	Memória liberada (8x)
20%	1,18x	1,21x	17,5%
40%	1,43x	1,54x	35,0%
60%	1,82x	2,11x	52,5%
80%	2,50x	3,33x	70,0%

Leitura honesta da economia: para um serviço memória-bound, liberar $f \cdot (1 - 1/r)$ da RAM significa, na prática, ou caber mais dado/tenants no mesmo servidor (o multiplicador acima), ou precisar de proporcionalmente menos servidores para o mesmo dado. O efeito em dinheiro é amplificado pelo momento — com a DRAM de servidor cara e escassa, cada GB liberado vale mais do que valia há um ano. Mas vale repetir as fronteiras: isso se aplica ao anel de CPU, armazenamento e telemetria; o grosso do capex de IA é GPU para treino e inferência, que o Smart2Raw quase não toca, e cargas de acesso aleatório intenso ou cálculo denso não se beneficiam.

Parte VIII — Fechamento

Limites conhecidos, síntese e a frase que resume tudo.

33. Limitações e trabalho futuro

Em nome da honestidade técnica que guiou todo o projeto, as fronteiras conhecidas:

- **NEON e big-endian.** Validados em ambiente emulado fiel, mas ainda sem o selo final em silício ARM e em host big-endian físico.
- **SIMD explícito.** Cobre a soma de tipos pequenos (u8/u16); as demais reduções dependem do autovetorizador.
- **Group-by.** Limitado a chaves de 8 bits (até 256 grupos). Chaves maiores exigiriam um esquema radix de dois passos.
- **Inferência.** A biblioteca acelera reduções de apoio (como a correção de zero-point), mas não o GEMM em si; um micro-kernel de produto-ponto int8 (via VNNI) seria o próximo passo para tocar o cálculo.
- **Floats.** Fora de escopo por design; dados em ponto flutuante precisam ser quantizados externamente.
- **Largura por bloco (PFOR).** Implementada e testada com e sem sinal (S2RBlocked, 3,7× medido), e com soma por bloco acelerada por SIMD. O que resta: serialização do formato por bloco e um modo sub-byte (int4) para tocar pesos de LLM abaixo de int8.

As direções naturais incluem: um produto-ponto int8 com VNNI para aproximar um micro-kernel de GEMM; group-by de propósito geral por radix; uma política de auto-cura acionada automaticamente na remoção; e um caminho NEON para as demais reduções no ARM.

34. Síntese: mecanismo, efeito e ganho medido

A tabela amarra cada mecanismo de arquitetura ao efeito microarquitetural que ele explora e ao ganho que medimos.

Mecanismo	Efeito explorado	Ganho medido
Classe adaptativa	menos bytes por valor	75–87% de memória
Densidade de cache	mais valores por linha	min/max 8–10×
vpsadbw / SIMD	soma horizontal em HW, sem widening	soma u8 2,8–10×
Histograma 4-tabelas	quebra dependência read-modify-write	3,6–4× (dados enviesados)
Correção zero-point	redução, não matmul	2,3× (int8 4096×4096)
Penhasco de cache	working set cabe em cache mais rápido	até 22,8× (scan de máximo)
Largura por bloco (PFOR)	outlier infla só o próprio bloco	3,7× (0,01% outliers)
Soma por bloco SIMD	vpsadbw no tipo de cada bloco	~6,7× (zero-point int8)
Zone-map pela classe	pula colunas sem ler dados	94,5% de banda evitada
mmap zero-copy	evita cópia para a RAM	IO/startup na borda
Auto-cura de largura	encolhe após remover outlier	recicla memória

O fio condutor é único: cada peça de arquitetura existe para mover menos bytes ou para deixar o hardware fazer mais trabalho por instrução. É a mesma frase do projeto, agora visível no nível da máquina.

35. Encerramento

O Smart2Raw é, em essência, uma mudança de mentalidade sobre armazenamento numérico: em vez de assumir o tipo mais largo por segurança, deixa-se o sistema escolher o tipo mais eficiente que preserva os dados e operar diretamente nele. As inovações descritas — correções de correção, aritmética lazy-carry, SIMD com dispatch, I/O portátil, mmap zero-copy, largura bidirecional e group-by no formato compacto — estendem essa filosofia da estação de trabalho ao dispositivo de borda, sempre com a disciplina de só afirmar o que foi medido.

A frase que resume o projeto permanece: classifique uma vez, opere sempre no formato mais compacto.